CAT 1 TECHNICAL DOCUMENTATION

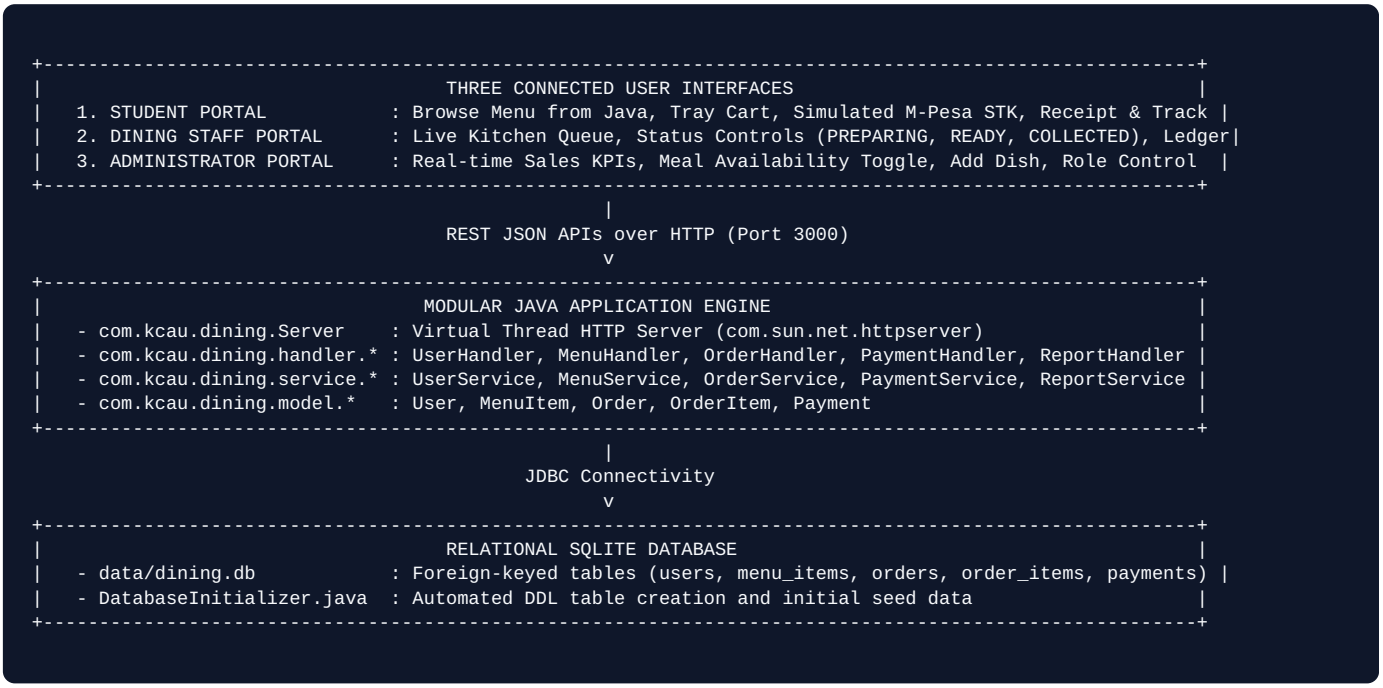
KCAU Smart Dining System

Complete technical guide covering modular Java backend architecture, SQLite relational persistence, three connected user interfaces, and Vercel edge deployment.

INSTITUTION	AUTHOR / REPOSITORY	BACKEND ENGINE	DATABASE & HOST
KCA University (KCAU)	solomon476 / KCAU-SMART-DINING-SYSTEM	Java 25 (Virtual Threads)	SQLite (data/dining.db) & Vercel

1. System Architecture & CAT 1 Design

The **KCAU Smart Dining System** is engineered according to Continuous Assessment Test 1 (CAT 1) requirements. It transitions client-only logic to a high-performance, modular Java backend that connects directly to a persistent SQLite relational database while delivering three interconnected portal experiences.



Dual Deployment Model: When executed locally, the application runs on the modular Java backend with SQLite persistence on `http://localhost:3000`. When deployed to **Vercel** (`https://kca-cafeteria.vercel.app`), the frontend leverages the resilient client-side `LocalDB` layer so the interactive demo works seamlessly in serverless edge environments.

Modular Java Architecture

Strictly partitioned into Model, Database, Service, Handler, and Server packages with virtual thread concurrency.

ACID Relational Storage

SQLite persistence ensures all accounts, food catalogs, meal tickets, and payment ledger entries survive restarts.

Role-Based Access Control

Three explicit user roles: STUDENT, STAFF, and ADMIN with cryptographic JWT authentication.

Dynamic Availability & Analytics

Sold out items toggle dynamically; Java calculates real-time revenue and dish popularity rankings.

2. Relational Database Schema (SQLite)

Data persistence is managed via `data/dining.db` with foreign-key constraints and ACID transaction guarantees. `DatabaseInitializer.java` automatically validates the schema and seeds default data upon first boot.

Table Name	Columns & Primary/Foreign Keys	Description & Role
`users`	`id` (TEXT PK) `name` (TEXT), `student_id` (TEXT UNIQUE) `password_hash` (TEXT), `role` (TEXT) `created_at` (TIMESTAMP)	Stores registered cafeteria users. The `role` column classifies accounts into `student`, `staff`, or `admin`. Passwords use PBKDF2 salt/hashing.
`menu_items`	`id` (TEXT PK) `name` (TEXT), `category` (TEXT) `price` (REAL), `description` (TEXT) `image` (TEXT), `available` (INTEGER/BOOLEAN)	Master catalog of campus dishes. The `available` flag (1=In Stock, 0=Sold Out) is controlled by Administrators and dynamically renders on student menus.
`orders`	`order_id` (TEXT PK) `user_id` (TEXT FK -> users.id) `customer_name` (TEXT) `total_amount` (REAL) `order_status` (TEXT) `created_at` (TIMESTAMP)	Kitchen meal tickets. Status progresses through `PENDING` -> `PREPARING` -> `READY` -> `COLLECTED`. Handled via atomic SQL transactions.
`order_items`	`id` (INTEGER PK AUTOINCREMENT) `order_id` (TEXT FK -> orders.order_id) `food_id` (TEXT), `food_name` (TEXT) `quantity` (INTEGER), `unit_price` (REAL)	Line item breakdown for each placed meal ticket. Links dishes ordered, counts, and unit prices for accurate kitchen display and analytics aggregation.
`payments`	`payment_id` (TEXT PK) `order_id` (TEXT FK -> orders.order_id) `method` (TEXT), `amount` (REAL) `reference` (TEXT UNIQUE) `status` (TEXT), `created_at` (TIMESTAMP)	Cafeteria transaction ledger. Records simulated M-Pesa STK push, Airtel Money, Card, or Counter Cash payments linked directly to order IDs.

Default CAT 1 Seed Accounts

Portal Role	Student / Staff ID	Full Name	Default Password	Portal Privileges
STUDENT	`KCA/1234/23`	Alex Kiprop	`password123`	Browse menu, cart tray, checkout, view receipt, track live ticket
STAFF	`STAFF/001`	Chef James Omondi	`password123`	Live kitchen display queue, status buttons, payment ledger
ADMIN	`ADMIN/001`	Dr. Jane Muthoni	`password123`	Sales metrics, toggle item availability, add dish, user role control

3. Backend Package & Code Architecture

The backend code is organized under package `com.kcau.dining` adhering to separation of concerns:

3.1 Package Organization

```
src/main/java/com/kcau/dining/
├── Server.java                      # Main server launcher with route registry & virtual threads
├── database/
│   ├── DatabaseConnection.java      # SQLite JDBC connection pool manager (data/dining.db)
│   └── DatabaseInitializer.java     # DDL table creation and initial seed data injector
├── model/
│   ├── User.java                    # Account entity with studentId, role, and password hash
│   ├── MenuItem.java                # Food catalog item with price, category, and availability
│   ├── Order.java                   # Order master with status lifecycle & line items list
│   ├── OrderItem.java               # Individual dish line item with count & unit price
│   └── Payment.java                 # Transaction entity with payment method, amount, and ref
├── service/
│   ├── UserService.java             # Account registration, PBKDF2 authentication, role updates
│   ├── MenuService.java             # Catalog retrieval, dish addition, availability toggling
│   ├── OrderService.java            # ACID order creation, status progression, and queries
│   ├── PaymentService.java          # Payment transaction recording and ledger queries
│   └── ReportService.java           # SQL aggregate sales summary & item popularity metrics
└── handler/
    ├── HttpHelper.java              # CORS, JSON serialization, and HMAC-SHA256 JWT tokens
    ├── UserHandler.java             # Endpoints: /api/signup, /api/login, /api/me, /api/users
    ├── MenuHandler.java             # Endpoints: /api/menu, /api/menu/availability, /api/menu/edit
    ├── OrderHandler.java            # Endpoints: /api/orders, /api/orders/my, /api/orders/status
    ├── PaymentHandler.java          # Endpoints: /api/payments
    └── ReportHandler.java           # Endpoints: /api/reports/summary, /api/reports/sales
```

3.2 Backend Service Implementation Details

- **OrderService (Transactional ACID Order Creation):**

```
// Atomic Order & OrderItem insertion within a single SQLite transaction
conn.setAutoCommit(false);
try (PreparedStatement orderStmt = conn.prepareStatement(insertOrderSql);
     PreparedStatement itemStmt = conn.prepareStatement(insertItemSql)) {
    // 1. Insert master order row
    orderStmt.setString(1, order.getOrderId());
    orderStmt.setString(2, order.getUserId());
    orderStmt.setDouble(4, order.getTotalAmount());
    orderStmt.executeUpdate();

    // 2. Batch insert individual order items
    for (OrderItem item : order.getItems()) {
        itemStmt.setString(1, order.getOrderId());
        itemStmt.setString(2, item.getFoodName());
        itemStmt.setInt(3, item.getQuantity());
        itemStmt.setDouble(4, item.getUnitPrice());
        itemStmt.addBatch();
    }
    itemStmt.executeBatch();
    conn.commit(); // Fully persisted atomically
}
```

- **ReportService (SQL Data Aggregation):** Calculates real-time total revenue, order count by status, average ticket value, and dishes ranked by quantity sold:

```
SELECT oi.food_name, SUM(oi.quantity) AS qty, SUM(oi.quantity * oi.unit_price) AS rev
FROM order_items oi JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_status != 'CANCELLED'
GROUP BY oi.food_name ORDER BY qty DESC;
```

- **MenuService (Dynamic Availability Control):** Toggles dish availability via `UPDATE menu_items SET available = ? WHERE id = ?`, immediately affecting student menu rendering.

4. REST API Endpoint Reference

All HTTP endpoints exchange standard JSON payloads and support Cross-Origin Resource Sharing (CORS):

Method	URI Endpoint	Request Body	Functionality & Status
`POST`	`/api/signup`	`{ name, studentId, password, role }`	Validates fields, creates user in SQLite with PBKDF2 hash. Returns `201 Created`.
`POST`	`/api/login`	`{ studentId, password }`	Authenticates credentials, returns signed JWT token with role. Returns `200 OK`.
`GET`	`/api/me`	`Bearer <token>` Header	Validates active JWT session and returns user identity. Returns `200 OK`.
`GET`	`/api/users`	*None (Admin)*	Returns list of all registered accounts and current roles. Returns `200 OK`.
`PUT`	`/api/users/role`	`{ userId, role }`	Promotes or updates account role to `student`, `staff`, or `admin`. Returns `200 OK`.
`GET`	`/api/menu`	*Optional ?category=...*	Returns complete menu catalog loaded from SQLite with availability flags. Returns `200 OK`.
`POST`	`/api/menu`	`{ name, category, price, desc, image }`	Adds a new dish to the cafeteria database. Returns `201 Created`.
`PUT`	`/api/menu/availability`	`{ foodId, available: boolean }`	Toggles dish status between In Stock and Sold Out in real time. Returns `200 OK`.
`POST`	`/api/orders`	`{ userId, customerName, totalAmount, items }`	Creates order ticket in SQLite with line items in an atomic transaction. Returns `201 Created`.
`GET`	`/api/orders`	*None (Staff/Admin)*	Retrieves all kitchen queue orders with item breakdowns. Returns `200 OK`.
`PUT`	`/api/orders/status`	`{ orderId, status }`	Transitions order status (`PREPARING`, `READY`, `COLLECTED`). Returns `200 OK`.
`POST`	`/api/payments`	`{ orderId, method, amount, reference, status }`	Records simulated payment transaction in payments table. Returns `201 Created`.
`GET`	`/api/payments`	*None (Staff/Admin)*	Returns full cafeteria transaction ledger. Returns `200 OK`.
`GET`	`/api/reports/summary`	*None (Admin)*	Calculates revenue, order counts by status, and registered user total. Returns `200 OK`.
`GET`	`/api/reports/sales`	*None (Admin)*	Returns dishes ranked by servings sold and total revenue earned. Returns `200 OK`.

5. Frontend Architecture & Three Connected Interfaces

The frontend is structured as a reactive Single Page Application (SPA) across `public/index.html`, `public/style.css`, and `public/app.js`:

5.1 The Three Connected Portals

- **1. Student Portal (`#student-menu`):**
  - Dishes loaded dynamically from Java SQLite backend (`GET /api/menu`).
  - Displays category filter pills (Breakfast, Lunch, Drinks, Fruits) and live search input.
  - Items marked `available = false` render with a red **SOLD OUT** badge and disabled Add button.
  - Floating Cart Dock tracks quantity and total price.
  - Checkout modal supports simulated M-Pesa STK push, Airtel Money, Card, and Cash at Counter.
  - Digital receipt generates Order ID and reference code.
  - Active Order Tracking banner polls status changes from the kitchen.
- **2. Dining Staff Portal (`#kitchen-dashboard`):**
  - Live Orders Queue displaying order cards with customer name, items list, and timestamps.
  - Filter pills: All, Pending, Preparing, Ready, Collected.
  - Status progression buttons: `Start Preparing 🍳` -> `Mark Ready 🔔` -> `Mark Collected ✓`.
  - Payment ledger tab displays transaction references and amounts.
  - Auto-polls the Java backend every 4 seconds.
- **3. Administrator Portal (`#admin-dashboard`):**
  - KPI summary cards: Total Revenue, Total Orders, Active in Kitchen, Registered Users.
  - Tab 1: Sales analytics with top-selling dishes and recent orders.
  - Tab 2: Meal catalog management with live Availability Toggle button and Add Dish modal.
  - Tab 3: User accounts table with role modification dropdown (Student / Staff / Admin).

5.2 Verification & How to Run

Execution Target	Command / URL	Expected Outcome
Run Java Backend Locally	`bash run-java.sh` or `npm start`	Starts Java server on port 3000, opens SQLite `data/dining.db`, seeds initial users and menu.
Test End-to-End	`http://localhost:3000/`	Test Student, Staff, and Admin flows using the Quick Demo Account chips on the login screen.
GitHub Repository	`https://github.com/solomon476/KCAU-SMART-DINING-SYSTEM`	Complete source code pushed with full Java models, services, database, and public assets.
Vercel Production Edge	`https://kca-cafeteria.vercel.app`	Live web application deployed with dual-layer fallback resilience.